COMPLETE TECHNICAL GUIDE · VERSION 1.0 · APRIL 2026

THE 7 LAYERS OF RAG

Retrieval-Augmented Generation

👥 Target: AI Engineers

📋 Prerequisites: LLMs · Embeddings · Vector DBs

🗓️ Last Updated: April 2026

FOUNDATION

Introduction to RAG

Retrieval-Augmented Generation (RAG) is an AI framework that improves the quality, accuracy, and reliability of Large Language Models (LLMs) by grounding their responses in external, authoritative knowledge bases. Instead of relying solely on the static data an LLM was trained on, a RAG system first *retrieves* relevant facts from a custom dataset and then *generates* an answer based exclusively on that retrieved context.

WHY RAG? (THE PROBLEM IT SOLVES)

Standard LLMs act like brilliant scholars who haven't read a new book since graduation. They suffer from static knowledge cutoffs, hallucinate when they lack information, and cannot access private company data. While **Fine-Tuning** teaches an LLM new information, it is computationally expensive and hard to update. **RAG** solves this by decoupling "knowledge" (vector DB) from "reasoning" (LLM), offering a cost-effective, accurate, and manageable enterprise AI solution.

KEY FEATURES

- **Dynamic Knowledge Access:** Pulls from live/updated databases without retraining models.
- **Contextual Grounding:** Dramatically reduces "hallucinations" by forcing answers directly from context.
- **Source Traceability:** Provides accurate, verifiable source citations.
- **Data Privacy & Security:** Queries highly sensitive data securely without exposing it to public training sets.

Real-Life Applications

🏢

Enterprise Knowledge

Internal chatbots searching Confluence, Drive, and Slack to answer policy or IT questions instantly.

> PRODUCTION REALITY:

🎧

Customer Support

AI agents ingesting manuals and tickets to provide highly specific user resolutions in real-time.

> PRODUCTION REALITY:

Scales to handle massive access-control lists (RBAC) so users only query docs they have permission to see.

Must strictly avoid hallucinating non-existent refund policies to prevent legal/financial liability.

⚖️

Legal & Medical

Systems allowing professionals to query case laws or journals with exact paragraph citations.

PRODUCTION REALITY:

Requires highly specialized chunking to avoid cutting critical "exceptions" away from primary rules.

💻

Codebase Copilots

Engineering tools indexing proprietary code so developers can ask how specific microservices interact.

PRODUCTION REALITY:

Uses fine-tuned code embedding models (like Voyage-code) to understand syntax rather than just semantics.

The 7 Layers at a Glance

Each layer transforms information in a specific way — together they build the complete RAG pipeline from raw documents to precise answers.

📥 OFFLINE INDEXING PIPELINE — LAYERS 1–4 · RUN ONCE (OR ON DOCUMENT UPDATE)

⚡ ONLINE QUERY PIPELINE — LAYERS 5–7 · RUN ON EVERY USER QUERY

#	LAYER NAME	FUNCTION	INPUT → OUTPUT
L1	Document Parsing & Ingestion	Convert raw documents to clean text	PDF/Word/HTML → Plain text

#	LAYER NAME	FUNCTION	INPUT → OUTPUT
L2	Chunking	Split text into manageable pieces	Long text → Small chunks
L3	Embedding Generation	Convert text to numerical vectors	Text chunks → Vector embeddings
L4	Vector Storage & Indexing	Store and index vectors for fast search	Vectors → Searchable index
L5	Query Processing & Retrieval	Convert query to vectors and search	User question → Relevant chunks
L6	Reranking & Filtering	Improve retrieval quality	Retrieved chunks → Re-ranked chunks
L7	Generation & Response Synthesis	Generate answer from retrieved context	Query + Chunks → Final answer

1

LAYER 01 · OFFLINE PIPELINE

Document Parsing & Ingestion

Transform raw, unstructured files — PDFs, Word docs, HTML, images — into clean, structured plain text with rich metadata that all downstream layers can consume.

Layer 1 handles every format: scanned PDFs need OCR, web pages need JS rendering, code files need AST parsing.

Parsing Techniques by Document Type

PDF Parsing

Scanned PDFs → OCR (Tesseract, AWS Textract). Text PDFs → pdfplumber, pypdf. Complex tables → Camelot, LayoutLM.

> **PRODUCTION REALITY:**
> Financial institutions use AWS Textract or Document AI to accurately extract tabular data from dense PDF annual reports.

Word (.docx)

Use python-docx to traverse elements. Extract tables cell-by-cell. Pull embedded images separately.

> **PRODUCTION REALITY:**
> Legal firms rely on structure-aware parsers to retain precise clause numbering and header hierarchies critical to contracts.

HTML / Web

Strip nav/ads with Readability.js. Dynamic JS content → Playwright or Puppeteer. Extract JSON-LD structured data.

> **PRODUCTION REALITY:**
> Enterprise intranets (SharePoint/Confluence) require parsers

Code Files

Preserve line numbers + language. Keep comments (they provide context). Parse AST for imports/dependencies.

> **PRODUCTION REALITY:**
> Tech companies strip out compiled binaries and lockfiles, explicitly indexing

that handle complex SSO/Auth gateways before content can be scraped.

internal documentation alongside AST graphs.

Real-World Output (JSON)

JSON · LAYER 1 OUTPUT


```
{ "chunks": [ { "text": "SECTION 1: VACATION POLICY\n\nAll full-time employees receive 20 days of paid vacation...", "metadata": { "source": "employee_handbook_2024.pdf", "page": 3, "section": "Vacation Policy", "last_modified": "2024-01-15", "author": "HR Department" } } ] }
```


Parsing Tools — Full Comparison & Enterprise Application

TOOL	FORMATS	OCR / TABLES	ENTERPRISE RATIONALE / WHY USE IT?
Unstructured.io	PDF, Word, HTML, PPT, Email	Yes / Yes	Standardizes ingestion pipelines for teams dealing with highly heterogeneous data (email + docs + web) simultaneously.
LangChain Loaders	100+ types (YouTube, Notion...)	Via integrations	Best for rapid prototyping and connecting to external SaaS apps (like Notion or Slack) via simple API wrappers.
pdfplumber	PDF only	No OCR / Yes (Cells)	Crucial for parsing text-native PDFs with complex financial or scientific tables where spatial boundaries matter.
AWS Textract / Google Doc AI	PDF, Images, TIFF	Advanced / Forms	Production workhorse for scanned documents. High cost but essential for processing thousands of scanned invoices or medical records.
Docling (IBM)	PDF, DOCX, PPTX, HTML	Yes / Yes	Provides deep layout understanding, ensuring reading order in multi-column research papers is preserved.

2

LAYER 02 · OFFLINE PIPELINE

Chunking (Text Segmentation)

Divide long documents into smaller, semantically coherent segments that fit within LLM context windows and enable precise retrieval. The critical balance between context and specificity.

✕ TOO-SMALL CHUNKS

Missing context around the answer · Answer split across chunks · Multiple retrievals needed · Poor semantic meaning per chunk

✕ TOO-LARGE CHUNKS

Retrieves irrelevant information · Dilutes relevant signal with noise · May exceed LLM context window · Slower embedding and search

11 Orchestrated Chunking Strategies

Here is a comprehensive breakdown of all chunking strategies used in RAG systems, spanning from simple heuristics to advanced LLM-driven architectures.

STRATEGY 1

Fixed-Size Chunking

Split text into chunks of an exact token or character count, regardless of content.

STRATEGY 2

Sentence-Based Chunking

Split at natural sentence boundaries (. ? !) and group 3–5 sentences per chunk.

- **How it works:** Every N tokens/characters → new chunk. Add 10–20% overlap to avoid losing context at boundaries.
- **Pros/Cons:** Fastest, predictable sizes *but* cuts sentences mid-thought, ignores meaning.
- **Best for:** Prototyping, plain homogeneous text.
- **Library:** `CharacterTextSplitter`

> **PRODUCTION REALITY:**
> Rarely used in modern production unless dealing with extremely uniform, unstructured logs.

- **How it works:** Use NLP sentence tokenizer (NLTK, spaCy), combine N sentences into one chunk.
- **Pros/Cons:** Preserves grammatical units *but* creates variable sizes; long sentences can create oversized chunks.
- **Best for:** News articles, FAQs, blog posts.
- **Library:** `NLTKTextSplitter` , `SpacyTextSplitter`

> **PRODUCTION REALITY:**
> Useful in Customer Support Q&A bots where answers are typically short, highly specific, and found within 1-2 exact sentences.

STRATEGY 3

Paragraph-Based Chunking

Each paragraph (separated by `\n\n`) becomes one chunk.

- **How it works:** Split on double newlines; merge very short paragraphs together.
- **Pros/Cons:** Naturally preserves topic boundaries *but* highly variable sizes (5 words vs 500 words).
- **Best for:** Well-structured reports, documentation.
- **Library:** `RecursiveCharacterTextSplitter (\n\n)`

STRATEGY 4 · RECOMMENDED DEFAULT

Recursive Character Chunking

Try splitting by the largest separator first, fall back to smaller ones.

- **How it works:** Tries `\n\n` → `\n` → `.` → space → character until chunks fit target size.
- **Pros/Cons:** Versatile, consistent sizes, respects structure *but* still not truly semantically aware.
- **Best for:** General-purpose starting point for most RAG pipelines.
- **Library:** `RecursiveCharacterTextSplitter`

STRATEGY 5 · HIGHEST QUALITY

Semantic Chunking

Split based on actual meaning shifts in the text — not characters or lines.

- **How it works:** Embed every sentence individually → Compute cosine similarity between consecutive sentences → Split where similarity drops below a threshold.
- **Pros/Cons:** Most semantically coherent chunks *but* slow and computationally expensive (not suited for real-time).
- **Best for:** High-quality production RAG, scientific papers, long-form content.
- **Library:** `SemanticChunker`

SEMANTIC SPLIT ALGORITHM (CONCEPTUAL)

```
S1→S2: 0.91 ← same topic, no split  S2→S3: 0.88 ← same topic, no split
S3→S4: 0.42 ← topic shift detected! ✂ SPLIT HERE  S4→S5: 0.90 ← new topic continues
```

> **PRODUCTION REALITY:**
> Essential for Legal and Medical domains. If a chunk splits a medical diagnosis from its list of side effects, the resulting RAG answer could be dangerous. Semantic boundaries prevent this.

STRATEGY 6

Document Structure-Aware

Use the document's own headings, sections, or tags as chunk boundaries.

- **How it works:** Parse Markdown headers, HTML tags, or PDF section titles as natural split points.
- **Pros/Cons:** Aligns perfectly with logical structure *but* requires highly structured docs.
- **Best for:** Markdown wikis, HTML docs, tagged PDFs, code repos.
- **Library:** `MarkdownTextSplitter` , `HTMLHeaderTextSplitter`

> **PRODUCTION REALITY:**
> The gold standard for Engineering teams indexing Github repos or Notion wikis.

STRATEGY 7

Token-Based Chunking

Split exactly on the model's tokenizer boundaries, not plain characters.

- **How it works:** Use `tiktoken` or HuggingFace tokenizers to count actual tokens.
- **Pros/Cons:** Highly precise for context windows *but* slower than character splitting.
- **Best for:** Production systems where strict token budget management is critical.

> Ensures an entire function or H2 policy section stays together.

- **Library:** `TokenTextSplitter`

STRATEGY 8 · ADVANCED

Agentic / Proposition-Based

An LLM rewrites the document into atomic, standalone factual statements before chunking.

- **How it works:** Feed paragraphs to an LLM → Extract self-contained propositions ("The refund window is 30 days") → Each proposition becomes a chunk.
- **Pros/Cons:** Highest retrieval precision possible *but* very expensive (one LLM call per paragraph) and slow.
- **Best for:** High-stakes Q&A (legal, compliance) where precision matters more than cost.

STRATEGY 9 · PRODUCTION STAPLE

Hierarchical / Parent-Child

Index small chunks for precision search, but pass larger parent chunks to the LLM for context generation.

- **How it works:** Search against Child chunks (128-256 tokens) → Pass mapped Parent chunk (512-1024 tokens) to LLM.
- **Pros/Cons:** Combines precision with context richness *but* requires more complex pipelines and double storage.
- **Best for:** Long documents where context around a fact matters.
- **Library:** `ParentDocumentRetriever`

STRATEGY 10

Sliding Window Chunking

Chunks overlap heavily — each new chunk slides forward by a fixed step.

- **How it works:** Chunk size 512, step 256 → 50% of every chunk is shared with the next.
- **Pros/Cons:** No info lost at boundaries *but* high storage overhead (2x chunks) and more retrieval noise.
- **Best for:** Medical records, contracts — where missing a boundary word is unacceptable.

- **Library:** Any splitter with high `chunk_overlap`

STRATEGY 11 · NEWEST APPROACH (2024)

Late Chunking

Embed the full document first, then split — preserving cross-chunk context in the embeddings.

- **How it works:** Pass the entire document through an embedding model (e.g., jina-embeddings-v3) → Use token-level output embeddings directly → Pool into chunks post-embedding.
- **Pros/Cons:** Eliminates context-loss because each chunk's embedding "knows about" the rest of the document *but* requires a long-context embedding model and lacks widespread tooling support.
- **Best for:** Documents where every section references concepts from other sections (e.g. legal contracts, research papers).
- **Library:** `chonkie` (Late chunker), Jina AI implementations.

Chunking Strategies — Quick Comparison Summary

Rule of thumb: Start with `RecursiveCharacterTextSplitter` (512 tokens, 10% overlap) → upgrade to `SemanticChunker` or `ParentDocumentRetriever` once you are actively measuring retrieval quality.

STRATEGY	SEMANTIC QUALITY	SPEED	COMPLEXITY	BEST DEFAULT?
Fixed-Size	⭐ Poor	⚡ Fastest	Simple	Dev only
Sentence-Based	⭐⭐ Moderate	⚡ Fast	Easy	✓
Paragraph-Based	⭐⭐ Moderate	⚡ Fast	Easy	✓
Recursive	⭐⭐⭐ Good	⚡ Fast	Easy	✅ Yes
Semantic	⭐⭐⭐⭐⭐ Best	🐢 Slow	Advanced	Production

STRATEGY	SEMANTIC QUALITY	SPEED	COMPLEXITY	BEST DEFAULT?
Structure-Aware	☆☆☆☆ Great	Fast	Moderate	Structured docs
Token-Based	☆☆ Moderate	Fast	Easy	Cost-critical
Agentic/Proposition	☆☆☆☆☆ Best	🦕 Slowest	Complex	High-stakes
Parent-Child	☆☆☆☆ Great	Medium	Moderate	✅ Production
Sliding Window	☆☆☆ Good	Medium	Easy	High-recall
Late Chunking	☆☆☆☆☆ Best	Medium	Advanced	Cross-ref docs

3

LAYER 03 · OFFLINE PIPELINE

Embedding Generation

Convert text chunks into dense numerical vectors that capture semantic meaning. These vectors are the mathematical "fingerprints" that enable similarity-based retrieval.

Each text chunk becomes a vector of hundreds or thousands of floating-point numbers representing its position in semantic space.

Embedding Generation Process

STEP	DESCRIPTION	EXAMPLE
1	Tokenize input text	`"Hello world" → [254, 567, 123]`
2	Pass through neural network (transformer encoder)	Attention layers extract meaning
3	Apply pooling (mean, CLS, or max)	Combine token embeddings into one vector
4	Normalize (L2 normalization)	Scale to unit length for cosine similarity
5	Output vector	`[0.12, -0.45, 0.78, ...]`

Embedding Models — Full Comparison

A comprehensive feature matrix across all major embedding models. MTEB scores are from the Massive Text Embedding Benchmark (higher = better).

MODEL	DIMS	CONTEXT (TOKENS)	MTEB SCORE	MULTILINGUAL	MATRYOSHKA
OpenAI text-embedding-3-small	1,536 (flex)	8,191	62.3	Yes	Yes
OpenAI text-embedding-3-large	3,072 (flex)	8,191	64.6	Yes	Yes
OpenAI text-embedding-ada-002	1,536	8,191	61.0	Yes	No
Cohere embed-english-v3.0	1,024	512	64.5	English only	No

MODEL	DIMS	CONTEXT (TOKENS)	MTEB SCORE	MULTILINGUAL	MATRYOSHKA
Cohere embed-multilingual-v3.0	1,024	512	62.9	Yes — 100+ langs	No
BGE-large-en-v1.5	1,024	512	63.98	English only	No
BGE-m3	1,024	8,192	64.8	Yes — 100+ langs	No
all-MiniLM-L6-v2	384	256	56.3	English only	No
all-mpnet-base-v2	768	384	57.8	English only	No
Voyage-2	1,024	4,000	62.4	Limited	No
Voyage-large-2-instruct	1,536	16,000	65.1	Limited	No
Voyage-code-2	1,536	16,000	—	No	No
Mistral Embed	1,024	8,192	60.5	Yes	No
Jina Embeddings v3	1,024	8,192	61.4	Yes — 89 langs	Yes
Nomic Embed Text v1.5	768	8,192	62.4	English only	Yes

Embedding Model Decision Matrix

YOUR PRIORITY	RECOMMENDED MODEL	WHY
🏆 Best overall quality	**OpenAI text-embedding-3-large**	Top MTEB score, Matryoshka, multilingual
💰 Best cost/quality balance	**OpenAI text-embedding-3-small**	Near-large quality at 6.5× cheaper
🌍 Multilingual (open source)	**BGE-m3**	100+ languages, 8K context, free
🔒 On-premise / no cloud	**BGE-large-en-v1.5 or BGE-m3**	Self-hostable, strong MTEB, free
⚡ Lowest latency (CPU)	**all-MiniLM-L6-v2**	384 dims, runs fast on CPU, free
💻 Code search / code RAG	**Voyage-code-2**	Specifically fine-tuned on code
📄 Long documents (>512 tokens)	**Voyage-large-2-instruct** or **BGE-m3**	16K and 8K context windows
$ Cheapest possible	**Jina v3 or all-MiniLM-L6-v2**	$0.02/1M (Jina) or Free (MiniLM)
🔎 Retrieval-optimized (English)	**Cohere embed-english-v3.0**	Trained specifically on retrieval tasks
📐 Variable storage needs	**OpenAI v3 or Nomic v1.5**	Matryoshka — truncate dims to trade quality for speed

Matryoshka Embeddings — What It Means

🪆 **Matryoshka Representation Learning (MRL)** trains the model so that the first N dimensions of a large embedding are themselves a high-quality smaller embedding. This means you can truncate `text-embedding-3-large` from 3,072 → 256 dims and still get strong retrieval quality — cutting storage and query costs dramatically with minimal accuracy loss.

TRUNCATED DIMS	RELATIVE MTEB SCORE	STORAGE VS FULL	USE CASE
3,072 (full)	100%	100%	Max accuracy, production search
1,536	~99.5%	50%	Balanced — recommended sweet spot
512	~98%	17%	High throughput with minimal quality loss
256	~95%	8%	Edge / latency-critical applications

Real-World Example

EMBEDDING OUTPUT (SIMPLIFIED)

```
# Input text chunk: "The company provides 20 days of paid vacation annually. Employees can carry over up to 5 unused days." # Output embedding (1536 dims — showing first 10): [0.023, -0.456, 0.789, -0.123, 0.345, -0.678, 0.901, -0.234, 0.567, -0.890, ...]
```

⚠ **Critical Rule:** Always use the **same embedding model** for indexing (Layer 3) and querying (Layer 5). Mismatched models produce incompatible vector spaces — retrieval will fail silently.

LAYER 04 · OFFLINE PIPELINE

4 Vector Storage & Indexing

Store embeddings in a specialized database optimized for fast similarity search. The index structure determines whether you can search millions of vectors in milliseconds or minutes.

Vector Database — Full Feature Comparison

A detailed breakdown of every major vector database across 12 key dimensions.

DATABASE	TYPE	OPEN SOURCE	MANAGED CLOUD	SELF-HOSTED	HYBRID S
Pinecone	Managed cloud	No	Yes	No	Yes (sparse-
Weaviate	Hybrid	Yes (BSD)	Yes (WCS)	Yes	Yes (BM built-in)
Qdrant	Hybrid	Yes (Apache 2.0)	Yes	Yes	Yes (spa vectors)
Milvus / Zilliz	Distributed	Yes (Apache 2.0)	Yes (Zilliz)	Yes	Yes
Chroma	Embedded	Yes (Apache 2.0)	No	Yes (in-process)	Partial (keywor
FAISS	C++ Library	Yes (Meta / MIT)	No	Yes	No

DATABASE	TYPE	OPEN SOURCE	MANAGED CLOUD	SELF-HOSTED	HYBRID S
pgvector	Postgres extension	Yes (MIT)	Via any Postgres host	Yes	Yes (FTS vector)
Elasticsearch	Full-text + vector	Source available	Yes (Elastic Cloud)	Yes	Yes — n BM25+A
Redis (Vector Search)	In-memory + vector	Yes (Redis CE)	Yes (Redis Cloud)	Yes	Partial
LanceDB	Embedded columnar	Yes (Apache 2.0)	Yes (LanceDB Cloud)	Yes	Yes (FTS ANN)

Vector DB Selection Guide

YOUR SCENARIO	BEST CHOICE	WHY
Starting a new project, want zero ops	Pinecone or Zilliz Cloud	Fully managed, no infra to worry about
Need full SQL-level metadata filtering	pgvector or Weaviate	SQL WHERE clauses / rich GraphQL
Already running Postgres	pgvector	No new infra — just add an extension
Already running Elasticsearch	Elasticsearch kNN	Native BM25 + dense vector hybrid
Billions of vectors, on-premise	Milvus	Distributed, battle-tested at scale
Local development only	Chroma or LanceDB	Embedded, zero setup, persists to disk

YOUR SCENARIO	BEST CHOICE	WHY
Sub-millisecond query latency	Redis Vector Search	In-memory, ~0.5ms at small scale
GPU research / custom indexes	FAISS	Direct control, IVF+PQ at any scale
Serverless / pay-per-query	LanceDB Cloud or Pinecone Serverless	No idle cost when traffic is bursty

Index Types — Full Comparison

INDEX TYPE	ALGORITHM	TIME COMPLEXITY	ACCURACY	MEMORY USAGE	BUILD
Flat (Brute Force)	Exact nearest-neighbor	O(n)	100% exact	High (all vectors in RAM)	Insta
IVF (Inverted File)	Cluster → search nearest buckets	O(√n)	95–99%	Medium	Minut
HNSW	Hierarchical navigable small world graph	O(log n)	95–99.9%	High (graph stored in RAM)	Minut Hours
PQ (Product Quantization)	Compress vectors into codes	O(n) but fast	90–95%	Very Low (8–16× compression)	Hours
IVF + PQ	Cluster then compress	O(√n) approx	90–96%	Very Low	Hours
LSH (Locality-Sensitive Hashing)	Hash similar vectors to same bucket	O(1) amortized	70–90% (high variance)	Low	Fast

INDEX TYPE	ALGORITHM	TIME COMPLEXITY	ACCURACY	MEMORY USAGE	BUILD
ScaNN	Anisotropic quantization (Google)	Sub-linear	Very High	Medium	Hours

Index Selection Guide

DATASET SIZE	RECOMMENDED INDEX	WHY
< 10,000 vectors	Flat (brute force)	Simple, exact, fast enough
10K – 1M vectors	HNSW	Best balance of speed and accuracy
1M – 100M vectors	IVF + PQ	Memory efficient, scalable
> 100M vectors	Milvus IVF_PQ	Distributed, production-scale

Stored Record Example

JSON · VECTOR DB RECORD

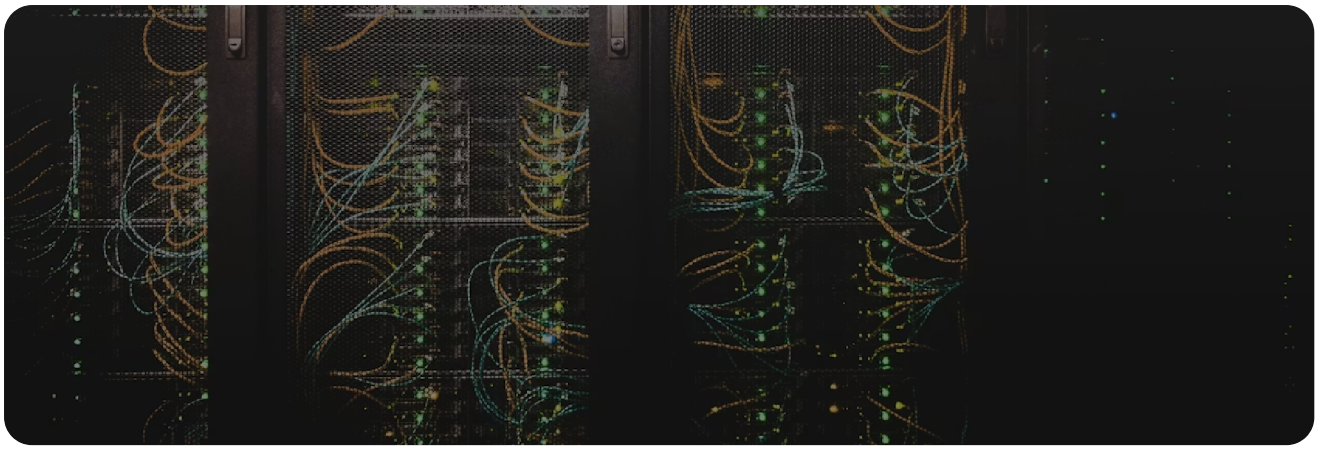
```
{ "id": "chunk_001234", "vector": [0.023, -0.456, 0.789, ...], // 1536 dimensions
"text": "The company provides 20 days of paid vacation annually...", "metadata":
{ "source": "employee_handbook_2024.pdf", "page": 3, "section": "Vacation
Policy", "last_modified": "2024-01-15", "doc_id": "HR-2024-001" } }
```

5

LAYER 05 · ONLINE PIPELINE

Query Processing & Retrieval

Convert a user's natural language question into an embedding, search the vector database, and return the most semantically similar text chunks. This is the first online layer.



Query Processing Steps

STEP	DESCRIPTION	EXAMPLE
1	Query understanding (optional)	Expand or rewrite for clarity
2	Embed query	Convert question to vector using same model as Layer 3
3	Search vector DB	Find top-K nearest neighbors
4	Apply metadata filters	Filter by date, source, department
5	Return results	Chunks + similarity scores + metadata

STRATEGY 1 · SIMPLEST

Simple Similarity Search

Embed query → Find top-K most similar vectors. Fast and straightforward but may miss contextually relevant, lexically different chunks.

STRATEGY 2 · RECOMMENDED

Hybrid Search (Keyword + Vector)

Combine vector similarity with BM25/TF-IDF keyword matching. Use Reciprocal Rank Fusion (RRF) to merge scores. Best of both worlds — captures semantic meaning AND exact keyword matches (product names, IDs, jargon).

STRATEGY 3

Multi-Query Retrieval

Generate multiple query variations using an LLM → Retrieve for each → Deduplicate and combine. Captures different angles of the user's question.

MULTI-QUERY EXAMPLE

```
# Original query: "What is the vacation policy?" # LLM-generated variations: "How many paid time off days do employees get?" "Annual leave policy for full-time staff" "Vacation accrual and carryover rules"
```

STRATEGY 4 · ADVANCED

HyDE — Hypothetical Document Embeddings

Generate a *hypothetical answer* to the query first → Embed that answer → Search with answer embedding (not query embedding). Often better aligned with answer-relevant documents.

STRATEGY 5 · ADVANCED

Self-Query Retrieval

LLM extracts structured metadata filters from natural language. *"Show me 2024 HR policies about vacation"* → automatically applies filter `year=2024, dept="HR", topic="vacation"` .

Real-World Retrieval Results

🔍 **Query:** "What is the maximum vacation carryover?" → Top 3 results returned:

RANK	SCORE	CHUNK TEXT
1	0.92	"Employees can carry over up to 5 unused vacation days to the next calendar year."
2	0.85	"Carryover requests must be submitted by December 15th."
3	0.71	"Any days beyond 5 will be forfeited at year end."

6

LAYER 06 · ONLINE PIPELINE

Reranking & Filtering

Improve retrieval quality by reordering, filtering, or pruning retrieved chunks using more sophisticated relevance signals than pure vector similarity. Turns good retrieval into great retrieval.

WITHOUT RERANKING

Vector similarity misses nuanced relevance · Top results may be only lexically similar · No diversity in results · Fixed K includes irrelevant chunks

WITH RERANKING

Cross-encoders capture deeper relevance · Semantic relevance properly prioritized · MMR ensures result diversity · Score threshold removes noise

The Reranking Pipeline

🔍 **Layer 5 Output: 20–100 chunks**

Fast approximate retrieval via vector similarity — casts a wide net

↓

🧠 **Cross-Encoder Reranking**

Scores each (query, chunk) pair — slow but highly accurate. Assigns 0–1 relevance score to each chunk.

↓

🎨 **MMR Diversity Optimization**

Balances relevance with diversity. Prevents 5 nearly identical chunks from dominating the context.

↓

✂️ **Score Threshold Filtering (≥ 0.5)**

Removes chunks below the relevance threshold. Sends only high-quality chunks to Layer 7.

↓

✅ **Layer 6 Output: 5–10 high-quality chunks**

Precisely relevant, diverse, clean — ready for generation

Bi-Encoder vs. Cross-Encoder

ASPECT	BI-ENCODER (LAYER 5)	CROSS-ENCODER (LAYER 6)
Processing	Query and chunk embedded separately	Query and chunk processed *together*
Speed	Fast (pre-computed)	Slow (computed at query time)
Accuracy	Moderate	High
Use case	Initial retrieval (millions → hundreds)	Reranking (hundreds → tens)

Before vs. After Reranking

BEFORE (VECTOR SCORE)	RANK	AFTER CROSS-ENCODER	FINAL SCORE
"Employees get 20 days of PTO."	1→1	"Employees get 20 days of PTO."	0.98
"Company provides 20 vacation days."	2→2	"Company provides 20 vacation days."	0.96
~~"Sick leave policy: 10 days."~~	3→5	"Vacation carryover: max 5 days."	0.85
"Vacation carryover: max 5 days."	4→3	"How to request time off in Workday."	0.62
"How to request time off."	5→4	❌ Sick leave — filtered out (0.12)	0.12

Reranking Models — Full Comparison

Cross-encoders score each (query, chunk) pair jointly — slower but far more accurate than bi-encoder similarity alone.

MODEL	ARCHITECTURE	SPEED	ACCURACY	CONTEXT (TOKENS)	MULTILI
Cohere Rerank v3	Cross-encoder (proprietary)	Fast (API)	Very High	4,096	Yes
Cohere Rerank-Multilingual-v3	Cross-encoder (proprietary)	Fast (API)	Very High	4,096	Yes — 1 langs
BGE-reranker-large	XLM-RoBERTa cross-encoder	Slow (GPU needed)	Very High	512	Yes
BGE-reranker-base	XLM-RoBERTa cross-encoder	Moderate	High	512	Yes

MODEL	ARCHITECTURE	SPEED	ACCURACY	CONTEXT (TOKENS)	MULTILI
BGE-reranker-v2-m3	BGE-M3 cross-encoder	Moderate	Very High	8,192	Yes — 1 langs
ms-marco-MiniLM-L-6-v2	MiniLM cross-encoder	Fast (CPU friendly)	Moderate	512	English
Jina Reranker v2	Cross-encoder	Fast (API)	High	8,192	Yes
Voyage Rerank-1	Cross-encoder (proprietary)	Fast (API)	Very High	4,000	Limited
Mixedbread mxbai-rerank-large-v1	DeBERTa-v3 cross-encoder	Moderate	Very High	512	English

Reranker Selection Guide

YOUR SCENARIO	BEST RERANKER
Production API with best quality	Cohere Rerank v3
Self-hosted, best quality	BGE-reranker-large or mxbai-rerank-large-v1
Multilingual + self-hosted	BGE-reranker-v2-m3
CPU only, fast inference	ms-marco-MiniLM-L-6-v2
Long documents (>512 tokens)	BGE-reranker-v2-m3 or Jina Reranker v2
Cheapest API option	Jina Reranker v2 ($0.07/1K)

7

LAYER 07 · ONLINE PIPELINE

Generation & Response Synthesis

The final layer. Retrieved chunks are combined with the user's query and passed to an LLM to generate a coherent, accurate, cited answer grounded entirely in your documents.

Prompt Template for Generation

RAG GENERATION PROMPT TEMPLATE

```
[SYSTEM] You are a helpful AI assistant. Answer based ONLY on the provided context. If the answer is not in the context, say "I don't have enough information to answer that." Cite sources using [1], [2], etc. Keep answer concise (max 3 sentences). [CONTEXT] [1] {chunk_1_text} (Source: {chunk_1_source}) [2] {chunk_2_text} (Source: {chunk_2_source}) [3] {chunk_3_text} (Source: {chunk_3_source}) [USER QUERY] {user_question} [ASSISTANT ANSWER]
```

Generation Strategies

📦

Stuff (Simple Concat)

🗺️

Map-Reduce

Concatenate all chunks → send to LLM. Simple, single call. Limited by context window size.

Map: answer from each chunk individually. Reduce: combine into final. Handles very long contexts but needs multiple LLM calls.

🔄

Refine (Iterative)

Start with first chunk → refine answer with each additional chunk sequentially. Builds progressively but slow.

🏆

Stuff + Rerank ← Best

After reranking, stuff top-K chunks into prompt. Best quality for typical RAG use cases.

LLM Selection for Generation

Choosing the right generation model is the biggest lever on both quality and cost.

MODEL	CONTEXT WINDOW	INPUT COST / 1M	OUTPUT COST / 1M	LATENCY	STRUCT
GPT-4o	128K	$5	$15	Moderate	Yes
GPT-4o mini	128K	$0.15	$0.60	Fast	Yes
GPT-4 Turbo	128K	$10	$30	Moderate	Yes
GPT-3.5 Turbo	16K	$0.50	$1.50	Fast	Yes

MODEL	CONTEXT WINDOW	INPUT COST / 1M	OUTPUT COST / 1M	LATENCY	STRUCT
Claude 3.5 Sonnet	200K	$3	$15	Moderate	Yes
Claude 3 Haiku	200K	$0.25	$1.25	Very Fast	Yes
Gemini 1.5 Pro	2M (!)	$3.50	$10.50	Moderate	Yes
Gemini 1.5 Flash	1M	$0.075	$0.30	Fast	Yes
Llama 3.1 70B	128K	Free (self-hosted)	Free	Slow (local GPU)	Partial
Llama 3.1 8B	128K	Free	Free	Fast (local)	Partial
Mistral Large	32K	$8	$24	Moderate	Yes
Mistral 7B	32K	Free	Free	Fast (local)	Partial

LLM Selection Guide for RAG

YOUR PRIORITY	BEST CHOICE	REASON
Best RAG quality overall	**GPT-4o or Claude 3.5 Sonnet**	Top faithfulness, citation accuracy
Best cost for high volume	**GPT-4o mini or Gemini 1.5 Flash**	$0.15–$0.075 input, solid quality
Longest context (whole codebase)	**Gemini 1.5 Pro (2M)**	2 million token window
No cloud / data-sovereignty	**Llama 3.1 70B (self-hosted)**	Open weights, on-premise
Lowest latency streaming	**Claude 3 Haiku or GPT-3.5 Turbo**	Fastest time-to-first-token
European data residency	**Mistral Large**	Hosted in EU, GDPR-friendly

Generation Parameters

PARAMETER	DESCRIPTION	RECOMMENDED FOR RAG
`temperature`	Randomness (0 = deterministic, 1 = creative)	**0.1–0.3 for factual Q&A**
`top_p`	Nucleus sampling threshold	0.9–1.0
`max_tokens`	Maximum response length	256–1024

Real-World End-to-End Output

FINAL RAG ANSWER WITH CITATIONS

```
# User Query: "How many vacation days do I get and can I carry over unused days?"
# Layer 7 Final Output (GPT-3.5 Turbo, temperature=0.2): """ According to the
employee handbook, full-time employees receive 20 days of paid vacation per
calendar year [1]. Additionally, you may carry over up to 5 unused days to the
next year, but you must submit your carryover request by December 15th [2]. """ #
Sources: [1] employee_handbook.pdf, page 3 [2] employee_handbook.pdf, page 5
```

Summary — The 7 Layers of RAG

Each layer answers one critical question and transforms data for the next.

1 **Document Parsing**

How do I extract clean text? → Plain text + metadata via Unstructured.io

2 **Chunking**

How do I split text? → Semantic chunks via LangChain splitters

3 **Embedding**

How do I convert text to numbers? → Vectors via OpenAI, Cohere, BGE

4 **Vector Storage**

How do I store and index? → HNSW index in Pinecone, Qdrant, or Chroma

5 **Retrieval**

How do I find relevant chunks? → Hybrid search (vector + BM25)

6 **Reranking**

How do I improve relevance? → Cross-encoder + MMR via Cohere Rerank

7 **Generation**

How do I produce the final answer? → Grounded, cited response via GPT-4, Claude, or Llama

---

Common Pitfalls & Solutions

LAYER 1

Garbled text from PDFs

✓ Use OCR for scanned PDFs. Test pdfplumber vs PyPDF2 on your documents.

LAYER 2

Chunks cut mid-sentence

✓ Use recursive splitter with sentence awareness and 10–20% overlap.

LAYER 2

Semantic drift in chunks

✓ Use semantic chunking instead of fixed-size character splits.

LAYER 5

Poor retrieval for rare terms

✓ Add hybrid search (keyword + vector). BM25 handles exact-match well.

LAYER 5

Outdated info in results

✓ Add recency boosting to retrieval. Filter by last_modified metadata.

LAYER 7

Hallucination despite RAG

✓ Lower temperature. Add explicit "I don't know" instruction to system prompt.

LAYER 7

Missing source citations

✓ Pass chunk IDs in prompt. Instruct model to cite using [1], [2] format.

LAYER 4

Slow query time

✓ Switch to HNSW index. Reduce K. Cache query embeddings.

LAYER 3 & 7

High cost

✓ Use smaller embedding model (BGE-small). Use GPT-3.5 or Haiku for generation.

Next Steps After Mastering the 7 Layers

Push further into advanced RAG patterns, evaluation, and production deployment.

🔄 Advanced RAG Patterns

Self-RAG, Corrective RAG (CRAG), and Adaptive RAG — systems that evaluate and improve their own retrievals.

📊 RAG Evaluation (RAGAS)

Measure faithfulness, answer relevancy, and context recall. Know whether your RAG system is actually working.

🚀 Production Deployment

Monitoring pipelines, embedding caches, A/B testing retrieval strategies, and cost optimization at scale.

🤖 Agentic RAG

Add tools, multi-step reasoning, and autonomous retrieval loops — beyond single-turn Q&A systems.

The 7 Layers of RAG — A Complete Technical Guide

Document Version 1.0 · April 2026 · Target: AI Engineers building RAG systems